



Universidade: Universidade Estácio de Sá
Curso: Desenvolvimento Full Stack
Turma: 9001

Campus: Polo alexandrino - Natal-RN
Disciplina: Programação Back-end Com Java
Semestre Letivo: 2026.1

Aluno: Guilherme Figueiredo Hermogenes Cardoso

Repositório Git:

<https://github.com/guicard321/Trabalho-Pratico-DGT2811-Desenvolvimento-Back--End-Corporativo-com-Java-E-Cloud.git>

Relatório de Acompanhamento de Prática 1º Procedimento

1. Título da Prática

Camadas de Persistência e Controle

2. Objetivo da Prática

Desenvolver uma aplicação web corporativa utilizando o padrão de arquitetura de três camadas, integrando tecnologias de persistência (JPA), regras de negócio (EJB) e interface web (Servlets), conectando-se a um banco de dados SQL Server via GlassFish.

3. Códigos Solicitados

Nota: Insira aqui os trechos do seu código:

- **Entidade:** *Produto.java*

```
package cadastroee.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import java.io.Serializable;

@Entity
public class Produto implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer idProduto;
```

```

private String nome;
private Integer quantidade;
private Float precoVenda;

public Produto() {
}

// Getters e Setters
public Integer getIdProduto() { return idProduto; }
public void setIdProduto(Integer idProduto) { this.idProduto = idProduto; }

public String getNome() { return nome; }
public void setNome(String nome) { this.nome = nome; }

public Integer getQuantidade() { return quantidade; }
public void setQuantidade(Integer quantidade) { this.quantidade = quantidade; }

public Float getPrecoVenda() { return precoVenda; }
public void setPrecoVenda(Float precoVenda) { this.precoVenda = precoVenda; }
}

```

- **Facade/EJB: *ProdutoFacade.java***

```

package cadastroee.controller;

import cadastroee.model.Produto;
import jakarta.ejb.Stateless;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;

/**
 *
 * @author guilh
 */
@Stateless
public class ProdutoFacade extends AbstractFacade<Produto> implements ProdutoFacadeLocal {

    @PersistenceContext(unitName = "CadastroEE-ejbPU")
    private EntityManager em;

    @Override
    protected EntityManager getEntityManager() {
        return em;
    }

    public ProdutoFacade() {
        super(Produto.class);
    }

}

```

- **Servlet: *ServletProduto.java***

```

package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import jakarta.ejb.EJB;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.List;

@WebServlet(name = "ServletProduto", urlPatterns = {"/ServletProduto"})
public class ServletProduto extends HttpServlet {

    // Letra D: Injeção do componente EJB no Servlet
}

```

```

@EJB
private ProdutoFacadeLocal facade;

protected void processRequest(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    // Define que a resposta será um HTML com acentuação correta
    response.setContentType("text/html;charset=UTF-8");

    // Busca a lista de produtos utilizando o nosso EJB Facade
    List<Produto> listaProdutos = facade.findAll();

    // Letra E: Modifica a resposta para apresentar os dados em formato de lista HTML
    try (PrintWriter out = response.getWriter()) {
        out.println("<!DOCTYPE html>");
        out.println("<html>");
        out.println("<head>");
        out.println("<title>Lista de Produtos - CadastroEE</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h1>Produtos Cadastrados</h1>");

        out.println("<ul>");
        if (listaProdutos == null || listaProdutos.isEmpty()) {
            out.println("<li>Nenhum produto encontrado no banco de dados.</li>");
        } else {
            for (Produto p : listaProdutos) {
                out.println("<li><strong>ID:</strong> " + p.getIdProduto() +
                    " | <strong>Nome:</strong> " + p.getNome() +
                    " | <strong>Qtd:</strong> " + p.getQuantidade() +
                    " | <strong>Preço:</strong> R$ " + p.getPrecoVenda() + "</li>");
            }
        }
        out.println("</ul>");

        out.println("</body>");
        out.println("</html>");
    }
}

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}
}

```

- **Configuração:** pom.xml e persistence.xml.

pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <artifactId>CadastroEE</artifactId>
        <groupId>com.minhaempresa</groupId>
        <version>1</version>
    </parent>
    <groupId>com.minhaempresa</groupId>
    <artifactId>CadastroEE-ejb</artifactId>
    <version>1</version>

```

```

<packaging>ejb</packaging>
<name>CadastroEE-ejb-1</name>

<properties>
  <maven.compiler.release>11</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <jakartaee>9.1.0</jakartaee>
</properties>

<dependencies>
  <dependency>
    <groupId>jakarta.platform</groupId>
    <artifactId>jakarta.jakartaee-api</artifactId>
    <version>${jakartaee}</version>
    <scope>provided</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.10.1</version>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-ejb-plugin</artifactId>
      <version>3.2.1</version>
      <configuration>
        <ejbVersion>3.2</ejbVersion>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

persistence.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<persistence version="1.0" xmlns="http://java.sun.com/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd">
  <persistence-unit name="CadastroEE-ejbPU" transaction-type="JTA">
    <jta-data-source>jdbc/loja</jta-data-source>
    <exclude-unlisted-classes>>false</exclude-unlisted-classes>
  </persistence-unit>
</persistence>

```

4. Resultados da Execução

- Print 1:

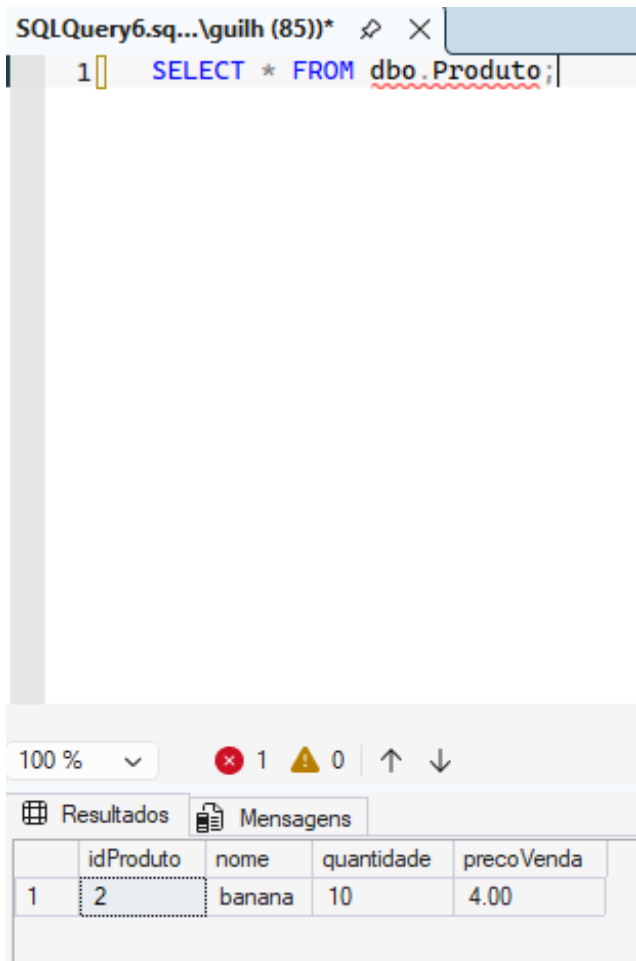
página web funcionando



Resultado servlet



- Print 2: O SQL Server Management Studio



5. Análise e Conclusão

Como é organizado um projeto corporativo no NetBeans?

Projetos corporativos no NetBeans são organizados em uma estrutura modular. Utiliza-se tipicamente o Maven (ant foi trocado para maven nas versões atuais) para gerenciar dependências e dividir o projeto em: **módulo EJB** (lógica de negócio/JPA), **módulo Web** (interface/Servlets).

Qual o papel das tecnologias JPA e EJB na construção de um aplicativo Web?

O JPA atua como a ponte entre o paradigma orientado a objetos do Java e o modelo relacional do banco de dados, automatizando o mapeamento (ORM). Já os EJBs fornecem um contêiner robusto para a lógica de negócio, gerindo automaticamente transações, controle de acesso e concorrência, permitindo que a aplicação Web se foque apenas na apresentação dos dados.

Como o NetBeans viabiliza a melhoria de produtividade ao lidar com JPA e EJB?

O NetBeans oferece ferramentas de automação (*Wizards*) que reduzem drasticamente o tempo de desenvolvimento. Por exemplo, a capacidade de gerar classes de entidade (*Entity Classes*) diretamente de tabelas do banco de dados, e a criação automática de *Session Beans* (*Facade*) para operações de CRUD, garantindo que o desenvolvedor siga as melhores práticas sem necessidade de codificação manual repetitiva.

O que são Servlets e como o NetBeans oferece suporte à construção deles?

Servlets são componentes Java que estendem as funcionalidades de um servidor web, permitindo responder a requisições HTTP dinamicamente. O NetBeans facilita a criação de Servlets através de assistentes que geram a estrutura básica (métodos `doGet`, `doPost`), além de configurar automaticamente os metadados necessários para o roteamento da aplicação, integrando-os diretamente ao servidor de aplicação (como o GlassFish).

Como é feita a comunicação entre os Servlets e os Session Beans do pool de EJBs?

A comunicação é realizada via **Injeção de Dependência**. O container EJB gerencia um *pool* de instâncias dos Beans. Quando um Servlet necessita de um EJB, ele utiliza a anotação `@EJB`. O contêiner intercepta essa referência e fornece uma instância pronta e configurada para o Servlet, permitindo a execução de métodos de negócio de forma segura e transacional.

Relatório de Acompanhamento de Prática 2º

Procedimento

1. Título da Prática

Interface Cadastral com Servlet e JSPs

2. Objetivo da Prática

Desenvolver a camada de apresentação de uma aplicação Web em Java (Jakarta EE), implementando o padrão de projeto *Front Controller* por meio de um *Servlet* (ServletProdutoFC). O objetivo é integrar páginas dinâmicas (JSPs) com a camada de negócio (EJB) e persistência (JPA), permitindo realizar operações de CRUD (Listar, Incluir, Alterar e Excluir) em um banco de dados relacional.

3. Códigos Solicitados neste Roteiro

Arquivo: *ServletProdutoFC.java*

```
package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import jakarta.ejb.EJB;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet(name = "ServletProdutoFC", urlPatterns = {"/ServletProdutoFC"})
public class ServletProdutoFC extends HttpServlet {

    @EJB
    private ProdutoFacadeLocal facade;

    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String acao = request.getParameter("acao");
        if (acao == null) {
            acao = "listar";
        }

        String destino = "ProdutoLista.jsp";

        try {
            switch (acao) {

                case "listar":
```

```

        request.setAttribute("produtos", facade.findAll());
        break;

    case "formIncluir":
        destino = "ProdutoDados.jsp";
        break;

    case "formAlterar":
        Integer idForm = Integer.parseInt(request.getParameter("id"));
        // Se Produto ficar vermelho, use Ctrl+Shift+I
        Produto produtoAlterar = facade.find(idForm);
        request.setAttribute("produto", produtoAlterar);
        destino = "ProdutoDados.jsp";
        break;

    case "excluir":
        Integer idExcluir = Integer.parseInt(request.getParameter("id"));
        Produto produtoExcluir = facade.find(idExcluir);
        if (produtoExcluir != null) {
            facade.remove(produtoExcluir);
        }
        request.setAttribute("produtos", facade.findAll());
        break;

    case "alterar":
        Integer idAlterar = Integer.parseInt(request.getParameter("id"));
        Produto produtoEdit = facade.find(idAlterar);

        if (produtoEdit != null) {
            produtoEdit.setNome(request.getParameter("nome"));
            // Se o seu preço for Double, mude a linha abaixo para Double.parseDouble
            produtoEdit.setPrecoVenda(Float.parseFloat(request.getParameter("preco")));
            produtoEdit.setQuantidade(Integer.parseInt(request.getParameter("quantidade")));
            facade.edit(produtoEdit);
        }
        request.setAttribute("produtos", facade.findAll());
        break;

    case "incluir":
        Produto novoProduto = new Produto();
        novoProduto.setNome(request.getParameter("nome"));
        // Se o seu preço for Double, mude a linha abaixo para Double.parseDouble
        novoProduto.setPrecoVenda(Float.parseFloat(request.getParameter("preco")));
        novoProduto.setQuantidade(Integer.parseInt(request.getParameter("quantidade")));

        facade.create(novoProduto);
        request.setAttribute("produtos", facade.findAll());
        break;
    }
} catch (Exception e) {
    System.err.println("Erro ao processar a ação " +acao + ": " + e.getMessage());
}

request.getRequestDispatcher(destino).forward(request, response);
}

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

@Override
public String getServletInfo() {
    return "Front Controller para gestão de Produtos";
}
}

```

Arquivo: *ProdutoLista.jsp*


```

<%@page import="java.util.List"%>
<%@page import="cadastroee.model.Produto"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Lista de Produtos</title>
<style>
body { font-family: Arial, sans-serif; margin: 20px; }
table { border-collapse: collapse; width: 100%; margin-top: 20px; }
th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
th { background-color: #f2f2f2; }
a { text-decoration: none; color: #0056b3; margin-right: 10px; }
a:hover { text-decoration: underline; }
.btn-novo { display: inline-block; padding: 10px 15px; background-color: #28a745; color: white;
margin-bottom: 15px; }
</style>
</head>
<body>
<h1>Produtos Cadastrados</h1>

<a href="ServletProdutoFC?acao=formIncluir" class="btn-novo">Incluir Novo Produto</a>

<table>
<tr>
<th>ID</th>
<th>Nome</th>
<th>Quantidade</th>
<th>Preço</th>
<th>Ações</th>
</tr>
<%

List<Produto> produtos = (List<Produto>) request.getAttribute("produtos");

if (produtos != null && !produtos.isEmpty()) {

    for (Produto p : produtos) {
%>
<tr>
<td><%= p.getIdProduto() %></td>
<td><%= p.getNome() %></td>
<td><%= p.getQuantidade() %></td>
<td><%= p.getPrecoVenda() %></td>
<td>
<a href="ServletProdutoFC?acao=formAlterar&id=<%= p.getIdProduto() %>">Alterar</a>
<a href="ServletProdutoFC?acao=excluir&id=<%= p.getIdProduto() %>" onclick="return
confirm('Excluir este produto?');">Excluir</a>
</td>
</tr>
<%
    }
} else {
%>
<tr>
<td colspan="5">Nenhum produto cadastrado.</td>
</tr>
<%
    }
%>
</table>
</body>
</html>

```

Arquivo: ProdutoDados.jsp

```

<%@page import="cadastroee.model.Produto"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Dados do Produto</title>
<style>

```

```

body { font-family: Arial, sans-serif; margin: 20px; }
.form-group { margin-bottom: 15px; }
label { display: inline-block; width: 100px; font-weight: bold; }
input[type="text"], input[type="number"] { width: 200px; padding: 5px; }
button { padding: 8px 15px; background-color: #0056b3; color: white; border: none; cursor: pointer; }
</style>
</head>
<body>
  <h1>Cadastro de Produto</h1>

  <%

    Produto produto = (Produto) request.getAttribute("produto");

    String acao = (produto == null) ? "incluir" : "alterar";
    String id = (produto != null && produto.getIdProduto() != null) ? produto.getIdProduto().toString() : "";
    String nome = (produto != null && produto.getNome() != null) ? produto.getNome() : "";
    String quantidade = (produto != null && produto.getQuantidade() != null) ?
produto.getQuantidade().toString() : "";
    String preco = (produto != null && produto.getPrecoVenda() != null) ? produto.getPrecoVenda().toString()
: "";

    String textoBotao = (produto == null) ? "Incluir Produto" : "Salvar Alteração";
  %>

  <form action="ServletProdutoFC" method="POST">

    <input type="hidden" name="acao" value="<%= acao %>">

    <input type="hidden" name="id" value="<%= id %>">

    <div class="form-group">
      <label>Nome:</label>
      <input type="text" name="nome" value="<%= nome %>" required>
    </div>

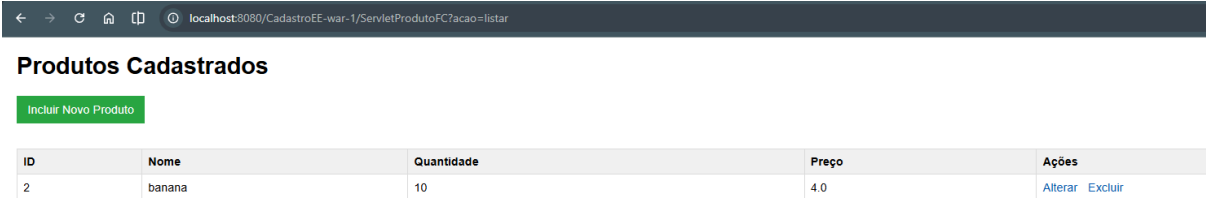
    <div class="form-group">
      <label>Quantidade:</label>
      <input type="number" name="quantidade" value="<%= quantidade %>" required>
    </div>

    <div class="form-group">
      <label>Preço (R$):</label>
      <input type="number" step="0.01" name="preco" value="<%= preco %>" required>
    </div>

    <button type="submit"><%= textoBotao %></button>
    <br><br>
    <a href="ServletProdutoFC?acao=listar">Voltar</a>
  </form>
</body>
</html>

```

4. Resultados da Execução



ID	Nome	Quantidade	Preço	Ações
2	banana	10	4.0	Alterar Excluir

- *Listagem Inicial de Produtos (acao=listar).*

Cadastro de Produto

Nome:

Quantidade:

Preço (R\$):

[Incluir Produto](#)

[Voltar](#)

- *Formulário de Inclusão preenchido.*

Produtos Cadastrados

[Incluir Novo Produto](#)

ID	Nome	Quantidade	Preço	Ações
2	banana	10	4.0	Alterar Excluir
3	Teclado Mecanico	2	280.0	Alterar Excluir

- *Tabela atualizada após a Inclusão.*

Cadastro de Produto

Nome:

Quantidade:

Preço (R\$):

[Salvar Alteração](#)

[Voltar](#)

- *Formulário de Alteração de um registro existente.*

← → ↺ 🏠 📄 ⓘ localhost:8080/CadastroEE-war-1/ServletProdutoFC

Produtos Cadastrados

[Incluir Novo Produto](#)

ID	Nome	Quantidade	Preço	Ações
2	banana	25	4.0	Alterar Excluir
3	Teclado Mecanico	2	280.0	Alterar Excluir

localhost:8080 diz

Excluir este produto?

[OK](#) [Cancelar](#)

- *Processo de Exclusão (alerta de confirmação).*

5. Análise e Conclusão

Como funciona o padrão Front Controller, e como ele é implementado em um aplicativo Web Java, na arquitetura MVC? O padrão *Front Controller* centraliza o tratamento de todas as requisições de um sistema web em um único ponto de entrada. Na arquitetura MVC (Model-View-Controller), ele atua como o **Controller principal**. Em aplicativos Web Java, é implementado tipicamente por meio de um único *Servlet* (como o *ServletProdutoFC*). Esse *Servlet* recebe todas as chamadas do cliente, analisa um parâmetro de comando (a variável *acao*), interage com o **Model** (regras de negócio e banco de dados via EJB/JPA) para processar os dados e, por fim, seleciona e encaminha a requisição para a **View** (páginas JSP) adequada para exibir a resposta.

Quais as diferenças e semelhanças entre Servlets e JSPs?

- **Semelhanças:** Ambas são tecnologias do lado do servidor (*Server-side*) da plataforma Java EE utilizadas para gerar páginas dinâmicas. Nos bastidores, o servidor de aplicação compila toda página JSP transformando-a em um *Servlet* antes da execução.
- **Diferenças:** O **Servlet** é uma classe Java que contém código HTML embutido em comandos de impressão. É voltado para atuar como *Controller*, lidando com lógica, segurança e roteamento. O **JSP (JavaServer Pages)** é um arquivo HTML que contém marcações e pequenos trechos de código Java embutidos. É voltado para atuar como *View*, focando na apresentação visual e na interface com o usuário.

Qual a diferença entre um redirecionamento simples e o uso do método forward, a partir do RequestDispatcher?

- **Redirecionamento Simples (response.sendRedirect):** Ocorre no lado do cliente. O servidor responde informando uma nova URL; o navegador faz uma *nova requisição HTTP* para esse endereço. A URL na barra de navegação muda e os dados da requisição original são perdidos.
- **Encaminhamento (RequestDispatcher.forward):** Ocorre inteiramente no lado do servidor. O *Servlet* repassa o processamento da requisição para outro recurso (como um JSP) de forma transparente para o cliente. A URL no navegador não muda, a requisição HTTP continua sendo a mesma e todos os dados e atributos permanecem disponíveis.

Para que servem parâmetros e atributos nos objetos HttpRequest?

- **Parâmetros (request.getParameter):** São informações enviadas **pelo cliente (navegador)** para o servidor, oriundas de formulários ou da própria URL (*Query String*). São imutáveis no escopo do servidor e transitam apenas como texto (*String*).
- **Atributos (request.setAttribute / getAttribute):** São informações criadas **pelo servidor** para compartilhar dados entre os componentes internos da aplicação durante o ciclo de vida de uma requisição. Permitem o tráfego de objetos Java complexos (como entidades ou listas) do *Controller* (*Servlet*) para a *View* (*JSP*).

Relatório de Acompanhamento de Prática 3º

Procedimento

1. Título da Prática

Melhorando o Design da Interface

2. Objetivo da Prática

Desenvolver e aprimorar a camada de apresentação de uma aplicação Web corporativa em Java (Jakarta EE). O objetivo central é integrar o padrão arquitetural *Front Controller* (via Servlet) a páginas JSP dinâmicas, aplicando o framework Bootstrap para garantir um design moderno, padronizado, estruturalmente independente e responsivo.

3. Códigos Solicitados neste Roteiro

Arquivo: ServletProdutoFC.java

```
package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import jakarta.ejb.EJB;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet(name = "ServletProdutoFC", urlPatterns = {"/ServletProdutoFC"})
public class ServletProdutoFC extends HttpServlet {

    @EJB
    private ProdutoFacadeLocal facade;

    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String acao = request.getParameter("acao");
        if (acao == null) {
            acao = "listar";
        }

        String destino = "ProdutoLista.jsp";

        try {
            switch (acao) {

                case "listar":
                    request.setAttribute("produtos", facade.findAll());
                    break;

                case "formIncluir":
                    destino = "ProdutoDados.jsp";
                    break;

                case "formAlterar":
                    Integer idForm = Integer.parseInt(request.getParameter("id"));
                    // Se Produto ficar vermelho, use Ctrl+Shift+I
            }
        }
    }
}
```

```

        Produto produtoAlterar = facade.find(idForm);
        request.setAttribute("produto", produtoAlterar);
        destino = "ProdutoDados.jsp";
        break;

    case "excluir":
        Integer idExcluir = Integer.parseInt(request.getParameter("id"));
        Produto produtoExcluir = facade.find(idExcluir);
        if (produtoExcluir != null) {
            facade.remove(produtoExcluir);
        }
        request.setAttribute("produtos", facade.findAll());
        break;

    case "alterar":
        Integer idAlterar = Integer.parseInt(request.getParameter("id"));
        Produto produtoEdit = facade.find(idAlterar);

        if (produtoEdit != null) {
            produtoEdit.setNome(request.getParameter("nome"));
            // Se o seu preço for Double, mude a linha abaixo para Double.parseDouble
            produtoEdit.setPrecoVenda(Float.parseFloat(request.getParameter("preco")));
            produtoEdit.setQuantidade(Integer.parseInt(request.getParameter("quantidade")));
            facade.edit(produtoEdit);
        }
        request.setAttribute("produtos", facade.findAll());
        break;

    case "incluir":
        Produto novoProduto = new Produto();
        novoProduto.setNome(request.getParameter("nome"));
        // Se o seu preço for Double, mude a linha abaixo para Double.parseDouble
        novoProduto.setPrecoVenda(Float.parseFloat(request.getParameter("preco")));
        novoProduto.setQuantidade(Integer.parseInt(request.getParameter("quantidade")));

        facade.create(novoProduto);
        request.setAttribute("produtos", facade.findAll());
        break;
    }
} catch (Exception e) {
    System.err.println("Erro ao processar a ação " +acao + ": " + e.getMessage());
}

request.getRequestDispatcher(destino).forward(request, response);
}

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

@Override
public String getServletInfo() {
    return "Front Controller para gestão de Produtos";
}
}

```

Arquivo: ProdutoLista.jsp

```

<%@page import="java.util.List"%>
<%@page import="cadastroee.model.Produto"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Lista de Produtos</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
    </head>

```

```

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
</head>

<body class="container">
  <br>
  <h1>Produtos Cadastrados</h1>

  <a href="ServletProdutoFC?acao=formIncluir" class="btn btn-primary m-2">Incluir Novo Produto</a>

  <table class="table table-striped">
    <thead class="table-dark">
      <tr>
        <th>ID</th>
        <th>Nome</th>
        <th>Quantidade</th>
        <th>Preço</th>
        <th>Ações</th>
      </tr>
    </thead>
    <tbody>
      <%
        List<Produto> produtos = (List<Produto>) request.getAttribute("produtos");
        if (produtos != null && !produtos.isEmpty()) {
          for (Produto p : produtos) {
            <tr>
              <td><%= p.getIdProduto() %></td>
              <td><%= p.getNome() %></td>
              <td><%= p.getQuantidade() %></td>
              <td><%= p.getPrecoVenda() %></td>
              <td>
                <a href="ServletProdutoFC?acao=formAlterar&id=<%= p.getIdProduto() %>" class="btn
btn-primary btn-sm">Alterar</a>
                <a href="ServletProdutoFC?acao=excluir&id=<%= p.getIdProduto() %>" class="btn btn-danger
btn-sm" onclick="return confirm('Excluir este produto?');">Excluir</a>
              </td>
            </tr>
          }
        } else {
          <tr>
            <td colspan="5">Nenhum produto cadastrado.</td>
          </tr>
        }
      <%>
    </tbody>
  </table>
</body>
</html>

```

Arquivo: ProdutoDados.jsp

```

<%@page import="cadastroee.model.Produto"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Dados do Produto</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
  </head>

  <body class="container">
    <br>
    <h1>Cadastro de Produto</h1>

    <%
      Produto produto = (Produto) request.getAttribute("produto");
      String acao = (produto == null) ? "incluir" : "alterar";
      String id = (produto != null && produto.getIdProduto() != null) ? produto.getIdProduto().toString() : "";
      String nome = (produto != null && produto.getNome() != null) ? produto.getNome() : "";
    <%>

```

```

String quantidade = (produto != null && produto.getQuantidade() != null) ?
produto.getQuantidade().toString() : "";
String preco = (produto != null && produto.getPrecoVenda() != null) ? produto.getPrecoVenda().toString()
: "";
String textoBotao = (produto == null) ? "Incluir Produto" : "Salvar Alteração";
%>

<form action="ServletProdutoFC" method="POST" class="form">
  <input type="hidden" name="acao" value="<%= acao %>">
  <input type="hidden" name="id" value="<%= id %>">

  <div class="mb-3">
    <label class="form-label">Nome:</label>
    <input type="text" name="nome" class="form-control" value="<%= nome %>" required>
  </div>

  <div class="mb-3">
    <label class="form-label">Quantidade:</label>
    <input type="number" name="quantidade" class="form-control" value="<%= quantidade %>" required>
  </div>

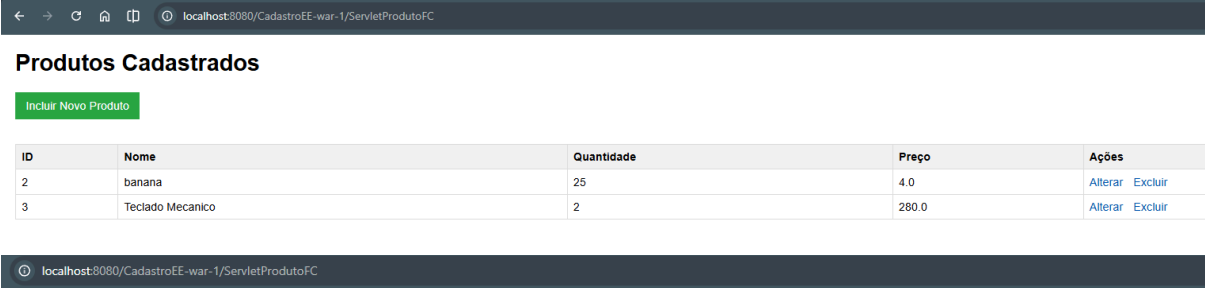
  <div class="mb-3">
    <label class="form-label">Preço (R$):</label>
    <input type="number" step="0.01" name="preco" class="form-control" value="<%= preco %>"
required>
  </div>

  <button type="submit" class="btn btn-primary"><%= textoBotao %></button>
  <br><br>
  <a href="ServletProdutoFC?acao=listar" class="btn btn-link">Voltar para a Lista</a>
</form>
</body>
</html>

```

4. Resultados da Execução

(Substitua os textos abaixo pelas capturas de tela que você fez do sistema rodando com o novo visual)



Produtos Cadastrados

Incluir Novo Produto

ID	Nome	Quantidade	Preço	Ações
2	banana	25	4.0	Alterar Excluir
3	Teclado Mecanico	2	280.0	Alterar Excluir

Produtos Cadastrados

Incluir Novo Produto

ID	Nome	Quantidade	Preço	Ações
2	banana	25	4.0	Alterar Excluir
3	Teclado Mecanico	2	280.0	Alterar Excluir

- **Figura 1: Tela de listagem de inicial de produtos com design atualizado (acao=listar).**

localhost:8080/CadastroEE-war-1/ServletProdutoFC?acao=formIncluir

Cadastro de Produto

Nome:

Quantidade:

Preço (R\$):

[Incluir Produto](#)

[Voltar para a Lista](#)

- **Figura 2: inclusão dinâmico, utilizando as classes de formulário do Bootstrap (acao=formIncluir).**

localhost:8080/CadastroEE-war-1/ServletProdutoFC

Produtos Cadastrados

[Incluir Novo Produto](#)

ID	Nome	Quantidade	Preço	Ações
2	banana	25	4.0	Alterar Excluir
3	Teclado Mecanico	2	280.0	Alterar Excluir
4	monitor 24 polegadas	5	1500.0	Alterar Excluir

- **Figura 3: inclusão bem-sucedida.**

localhost:8080/CadastroEE-war-1/ServletProdutoFC?acao=formAlterar&id=3

Cadastro de Produto

Nome:

Quantidade:

Preço (R\$):

[Salvar Alteração](#)

[Voltar para a Lista](#)

- **Figura 4: ação de alteração.**

localhost:8080/CadastroEE-war-1/ServletProdutoFC

Produtos Cadastrados

[Incluir Novo Produto](#)

localhost:8080 diz
Excluir este produto?

[OK](#) [Cancelar](#)

ID	Nome	Quantidade	Preço	Ações
2	banana	25	4.0	Alterar Excluir
3	Teclado Mecanico	2	250.0	Alterar Excluir
4	monitor 24 polegadas	5	1500.0	Alterar Excluir

- **Figura 5: Processo de exclusão**

5. Análise e Conclusão

Como o framework Bootstrap é utilizado? O Bootstrap é utilizado através da inclusão de suas bibliotecas de estilo (CSS) e comportamento (JavaScript) no documento HTML/JSP, geralmente referenciadas no cabeçalho (<head>) via CDN (Content Delivery Network). Uma vez que a biblioteca está vinculada ao projeto, o desenvolvedor utiliza o framework aplicando classes predefinidas diretamente nas tags HTML. Por exemplo, ao invés de escrever códigos CSS complexos manualmente, basta adicionar classes como `class="btn btn-primary"` em um botão ou `class="table table-striped"` em uma tabela para que os elementos herdem instantaneamente um design moderno e padronizado.

Por que o Bootstrap garante a independência estrutural do HTML? O Bootstrap promove uma clara separação entre o conteúdo (HTML) e a apresentação visual (CSS). No passado, os desenvolvedores utilizavam atributos estruturais dentro do HTML, como tabelas aninhadas complexas ou tags para definir fontes e cores, o que misturava a semântica dos dados com o design. Ao utilizar o Bootstrap, o documento HTML permanece focado em representar adequadamente a informação (textos, formulários, dados), enquanto toda a camada visual é aplicada por meio das classes. Isso garante que a estrutura lógica do HTML fique limpa e independente; caso o design precise ser radicalmente alterado no futuro, basta modificar o tema ou as classes, sem necessidade de reescrever a árvore DOM (Document Object Model).

Qual a relação entre o Bootstrap e a responsividade da página? A principal premissa do Bootstrap é ser *Mobile-First*, ou seja, projetado primeiramente para dispositivos móveis e depois escalado para telas maiores. A relação íntima entre o framework e a responsividade dá-se por meio do seu robusto sistema de *Grid* (baseado em Flexbox) e do uso intensivo de *Media Queries* nativas do CSS3 sob o capô. Classes como `container`, por exemplo, percebem automaticamente a largura da janela de exibição (*viewport*) do dispositivo — seja um monitor widescreen, um tablet ou um smartphone — e ajustam fluidamente as margens, tamanhos e o empilhamento dos elementos. Isso permite que a mesma aplicação Java Web seja perfeitamente acessível e esteticamente agradável em qualquer dispositivo, eliminando a necessidade de desenvolver múltiplas versões da mesma interface.